Speech API — Developer Documentation

Overview

The Speech API provides access to speech-to-text (STT) and text-to-speech (TTS) services using Microsoft Azure Cognitive Services.
It accepts a .wav audio file and returns text transcription, or accepts text and returns synthesized speech in .wav format.

Base URL

```
http://134.122.31.214/api/v1/
```

Authentication

No API keys are exposed through public requests.
Azure credentials are configured **server-side only** using environment variables:

- SPEECH_KEY
- SPEECH_REGION

Endpoints

1. Health Check

GET /health

Returns service status and timestamp.

Response — 200 OK

2. Speech-to-Text

POST /transcribe

Uploads a .wav file and returns its transcription.

Request Format

- multipart/form-data
- Field name: **audio**
- File type: **.wav**
- Max recommended length: 30 seconds

Successful Response — 200 OK

```
{
  "text": "Hello world from the speech to text endpoint.",
  "lengthCharacters": 48
}
```

Validation Rules

No file Uploaded	400 Bad Request
Not a .wav file	400 Bad Request
Missing Azure environment variables	500 Internal Error
Azure transcription failure	500 Internal Error

Error Examples

Invalid format

```
{
  "error": "Invalid file format. Only .wav audio files are accepted."
}
```

3. Text-to-Speech

POST /tts

Converts text input into synthesized .wav audio.

Request Format

- Application/json
- Body field: text (string)

```
{
  "text": "Hello from the text to speech endpoint."
}
```

Response

Content-Type: audio/wav

Response body = **binary audio stream**

This will appear in Postman as:

- Audio player

Postman Setup

1. GET /api/v1/health

Method: **GET**
URL: http://<SERVER_IP>/api/v1/health
Body: **None**
Headers: **None**

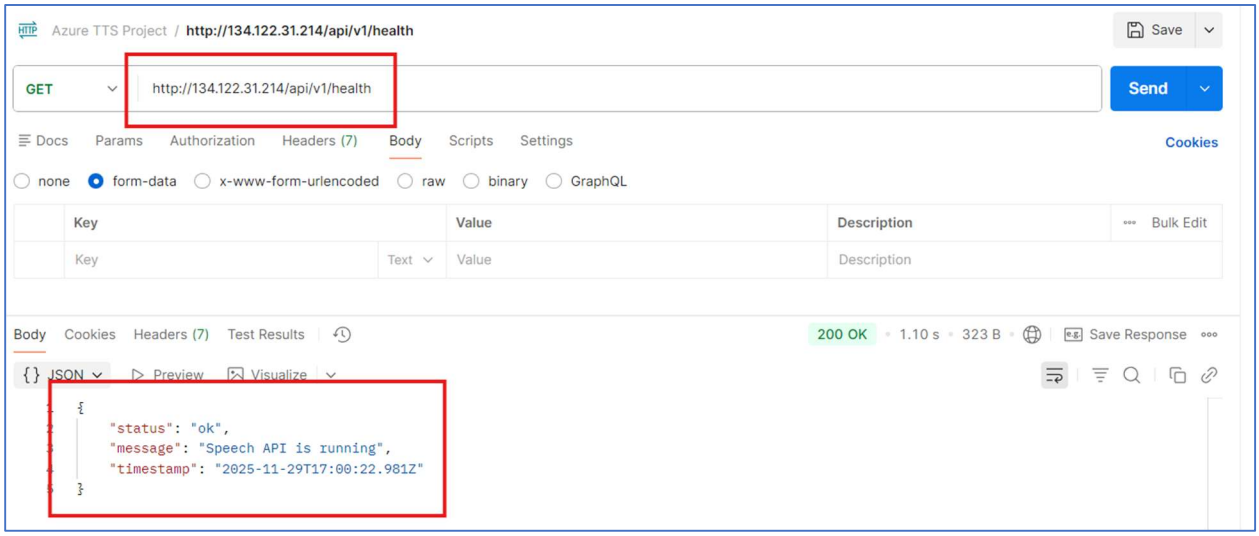

Figure 1: Successful Postman request

2. POST /api/v1/transcribe

Method: **POST**
URL: http://<SERVER_IP>/api/v1/transcribe
Body Type: **form-data**
Field:

- **audio → File (.wav)**

Headers:

- *(None required — Postman sets multipart boundaries automatically)*

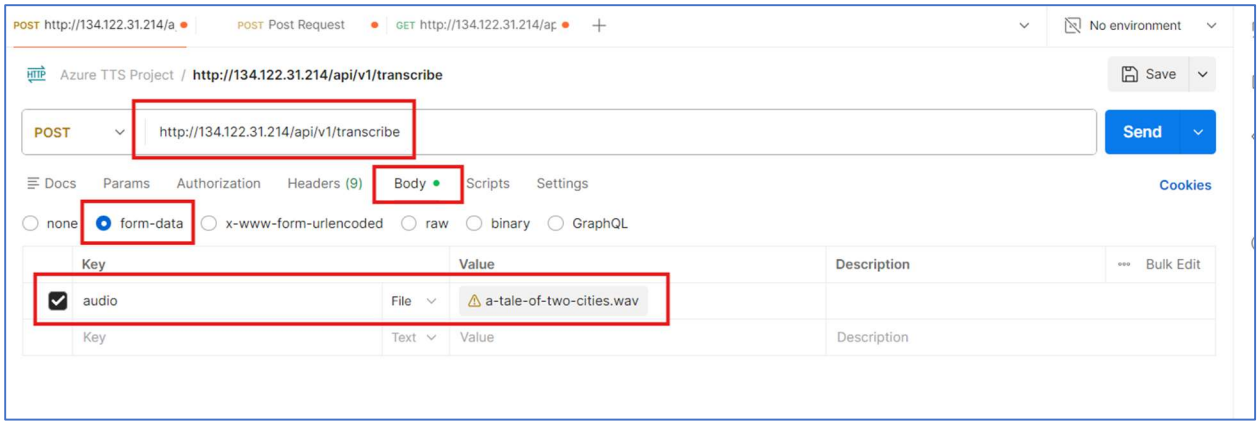

Figure 2: Postman configuration for transcribe testing

3. POST /api/v1/tts

Method: **POST**
URL: http://<SERVER_IP>/api/v1/tts
Body Type: **raw**
Format: **JSON**
Example Body:

```
{
  "text": "Hello from the text to speech endpoint."
}
```

Headers: Content-Type: application/json

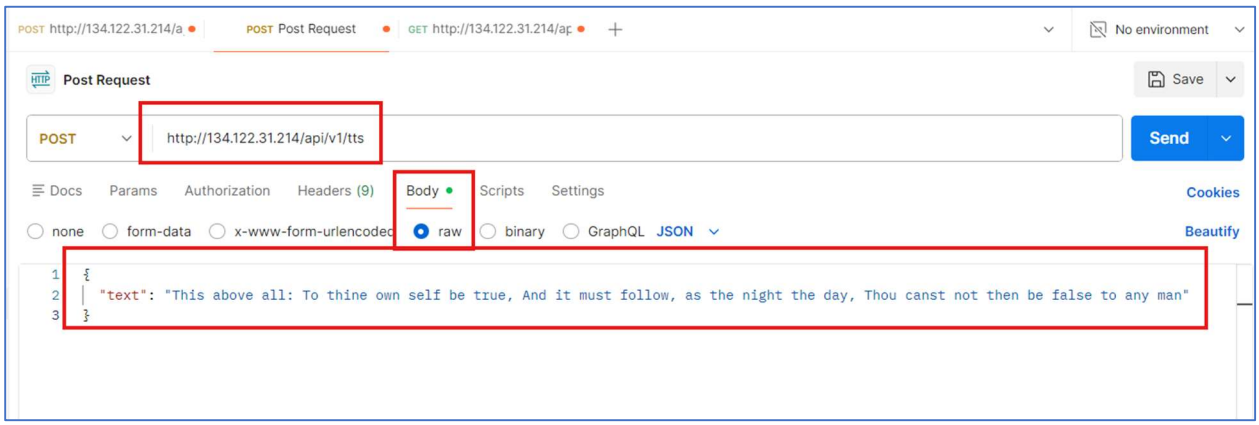

Figure 3: Postman configuration for text to speech testing

File Size & Duration Guidelines

Azure Speech is optimized for short utterances (3–30 seconds)

Longer files may:

- Take longer to process
- Return partial transcriptions
- Trigger cancellation messages

Status Codes

200	Success
400	Invalid input
404	Endpoint not found
500	Internal server or Azure SDK failure

Environment Configuration

```
Server requires:
SPEECH_KEY=<azure speech key>
SPEECH_REGION=<azure region, e.g. eastus>
PORT=80
```

Environment variables should be set in:

- .bashrc or .profile
- DigitalOcean droplets
- PM2 startup environment

Never stored in git.

Deployment Notes

- Node.js server runs on port 80
- Process manager: PM2
- Azure credentials remain server side
- Client never sees Azure keys or endpoint

Failure Modes & Debugging

400 bad request	No file or invalid input
Partial transcription	Audio too long or noisy
TTS returns empty audio	Malformed POST body
ECONNRESET	Azure service timeout
“Recognition cancelled: 1”	SDK internal cancellation

Best Practices

- Prefer short audio clips
- Use .wav not .mp3
- Use \n inside JSON string for line breaks
- Always validate uploads server-side
- Never expose Azure keys in front-end

API Usage Examples

1. Health Check — GET /health

cURL

```
curl http://<SERVER_IP>/api/v1/health
```

JavaScript (fetch)

```
const res = await fetch("http://<SERVER_IP>/api/v1/health");
const data = await res.json();
console.log(data);
```

Python (requests)

```
import requests

response = requests.get("http://<SERVER_IP>/api/v1/health")
print(response.status_code)
print(response.json())
```

2. Speech-to-Text — POST /transcribe

Uploads a .wav file and returns JSON with the transcription.

Important:

- “Body type: multipart/form-data”
- “Field name: audio”
- “File must be .wav”

cURL

```
curl -X POST "http://<SERVER_IP>/api/v1/transcribe" \
  -F "audio=@your-audio-file.wav
```

JavaScript (fetch, e.g., in a browser or Node with form-data)

```
// In a browser, assuming you have an <input type="file" id="fileInput" />
const fileInput = document.getElementById("fileInput");
const file = fileInput.files[0];

const formData = new FormData();
formData.append("audio", file);

const res = await fetch("http://YOUR_SERVER_IP/api/v1/transcribe", {
  method: "POST",
  body: formData
});

const data = await res.json();
console.log("Transcription:", data.text);
```

Python (requests)

```
import requests

url = "http://YOUR_SERVER_IP/api/v1/transcribe"

with open("your-audio-file.wav", "rb") as f:
    files = {"audio": f}
    response = requests.post(url, files=files)

print("Status:", response.status_code)
print("Response JSON:", response.json())
```

3. Text-to-Speech — POST /tts

Sends text and receives synthesized speech as a .wav file.

Important:

- “Body type: application/json”
- “Field: text (string)”
- “Response: binary WAV audio”

cURL

```
curl -X POST "http://YOUR_SERVER_IP/api/v1/tts" \
  -H "Content-Type: application/json" \
  -d '{"text": "Hello from the text to speech endpoint."}' \
  -o output.wav
```

This will save the synthesized audio to output.wav.

JavaScript (fetch)

```javascript
const res = await fetch("http://YOUR_SERVER_IP/api/v1/tts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    text: "Hello from the text to speech endpoint."
  })
});

// Get raw audio bytes
const arrayBuffer = await res.arrayBuffer();

// Optionally create a playable Blob (browser)
const audioBlob = new Blob([arrayBuffer], { type: "audio/wav" });
const audioUrl = URL.createObjectURL(audioBlob);
const audio = new Audio(audioUrl);
audio.play();
```

Python (requests)

```python
import requests

url = "http://YOUR_SERVER_IP/api/v1/tts"
payload = {"text": "Hello from the text to speech endpoint."}

response = requests.post(url, json=payload)

print("Status:", response.status_code)

# Save the audio to a file
with open("tts-output.wav", "wb") as f:
    f.write(response.content)

print("Saved synthesized audio to tts-output.wav")
```